Foundations of Artificial Intelligence
Joel Mattsson - Assignment 1
2024/2025

1. Problem Introduction

In this report, systematic problem solving approaches to the well-known game Sudoku will be examined from theoretical perspectives. Sudoku is composed of a 9x9 matrix such that 81 cells are divided by rows and columns. Initially, some boxes, but not all, are filled with digits ranging between 1 and 9, and the goal is to fill each cell with a digit. However, if a box, row or column has already a digit X, then that digit cannot appear in that specific row, column or box. This very concept is what brings logic and challenge to the game, forcing its players to continuously recall the numbers they previously filled in to identify potential inconsistencies.

2. Theoretical Background

Taking into consideration the rules of Sudoku, it becomes apparent that the root of the game is based on one's ability to keep track of not only the numbers filled in, but their relative board positions. Looking at this at a deeper level, there are primarily two factors determining an individual's success in Sudoku:

- **Memory:** Being able to remember and accurately recall the numbers in use for each row, cell and box
- **Strategy:** This is the more interesting part. The systematic strategy for filling in and changing pre-existing candidates based on the current circumstances of the board

Pursuing greatness in any endeavor requires developing supporting systems that facilitate repetitive behavior. In the context of Sudoku, a person that thinks in terms of a systematic strategy, or a heuristic, would have an advantage in regards to identifying patterns of numerical constraints. However, the human mind is limited and can only remember so much.

This then begs the question: Given a computer's superior memory and computational speed, how fast and accurately can a Sudoku puzzle be solved within, using search strategies?

A "search strategy", in this report, is referred to as a high-level description of the qualities needed for an algorithm to efficiently solve a specific type of problem. Considering that the scope of this assignment is target at Sudoku, a breakdown of its fundamental qualities is important:

- **1)** Search or node expansion: Efficiently propose new candidates that have potential of being in the final solution
- **2)** Heuristic: Disregard options not viable for a final solution before executing them. Selectively choose which branches of options that have actual potential
- **3)** Revert or optimize partial solutions: The initial assignment of cells is not always correct, and thus creates a need for multiple computations to find the complete solution

As it appears, a search strategy to determine which cell to target coupled with a combinatorial approach of assigning that cell or variable a tentative value, is fundamental to solve Sudoku. Breaking this down further, the problem to be solved is the assignments of all cells or variables. All of these assignments must not invalidate each other and therefore conform to the given constraints, and the values must also range within the specified domain. In other words, we need to find a combination such that each assignment of their values makes the entire puzzle valid, and this brings us to Constraint Satisfaction Problems.

2. 1 Constraint Satisfaction Problem

Hence, this problem can be formulated as a Constraint Satisfaction Problem (CSP):

- Variables: Generally embodies the variables that are assigned during runtime or as a round of the specific game plays out. These are imperative since it's their assignments that determine the success or failure of the algorithm to solve the puzzle. In this report, the variables are the cells, since the empty ones must be assigned.
- Domains: The restriction of allowed values for each variable. Fundamentally, the domain consists of a set of values that are potential variable assignments. In Sudoku, each cell has a value ranging between 1 and 9.
- Constraints: Controls the dependencies between the values of the variables and the current state of the partial solution. More specifically, it's a set of directives that reflect the rules of the associated game. For instance, one rule in Sudoku is for each digit to appear once in each box.
    - Direct Constraints: Explicitly defined relationships that restrict the possible values of variables. For example, in a CSP with variables **X** and **Y**, a direct constraint may be **X + Y = 5**. If one of these variables were to be resolved, there is only one possible value for the latter one to obtain, for their relationship to hold true. In Sudoku, …
        - *Each row must contain **ONE** instance of digits 1-9*

        - *Each column must contain **ONE** instance of digits 1-9*
        - *Each box or subgrid must contain **ONE** instance of digits 1-9*
    - Indirect Constraints: As opposed to direct constraints, these types of constraints aren't explicitly stated but rather derived from the direct ones. For instance, having three variables **A**, **B** and **C** with the direct constraints **A < B** and **B < C**, would entail an implicit and indirect constraint of **A < C**. In Sudoku…
        - Inferred Relationships: Placing a digit in a cell indirectly impacts the constraints of other cells in the same row, column and box. For instance, filling a cell with 8, that cell's row, column and box cannot contain another 8.
        - Constraint Propagation: If a row already contains 2, 4, 5, 6 and 8 you can infer that the only possible values are 1, 3 and 7. Furthermore, this information may lead to other implications for other columns and boxes.

Considering the combinatorial variable assignment in a CSP, a need for exploring nodes and putting each one through a trial period of assigning it a temporary value is the initial step. Now that we know the objectives of a searching strategy for Sudoku, and the nature of CSPs, let's observe "searching" as a general concept, and then viable options for choosing an approach.

2. 2 Searching Strategies

Searching is a broad and vital concept of programming. In simple terms, any case where a target element is needed that is contained in a dataset is considered as searching. Due to data structures, the elements in a dataset are usually sorted by certain qualities or attributes, and this ordering opens up possibilities for more efficient and systematic exploration to locate the target elements more quickly. This broad concept even covers algorithms known for their searching tendencies accompanied by their uncertainty of what exactly the target element is. It is still considered as searching as a certain element must be accessed from the dataset to complete the algorithm's mission. For instance, in Sudoku, the target is unknown. The value of the cell or variable can range from 1-9 and is restricted according to the indirect constraints of the board. In some cases, the candidate we are looking for is 3, 6 and 7. And sometimes the only valid target element is 7. Nevertheless, a searching strategy must be applied to find and filter the viable options of variable assignments to solve this CSP. Now that we have established that Sudoku requires active searching of cell values, the next question is which searching strategies are suitable to apply. Below, different potential options will be briefly discussed and evaluated based on their respective capabilities to solve Sudoku. The ones that turn out to be suitable will be utilized in the subsequent section "Approaches" to implement the approaches.

2. 2. 1 A* Search

A* is a pathfinding and graph traversal algorithm that makes informed decisions based on heuristics and previously acquired knowledge of visited nodes. The cost function can be expressed as follows, where **g(n)** is the cost from start node to the current node **n**, and **h(n)** is the estimated cost from node **n** to the goal node:

$$f(n) = h(n) + g(n)$$

Nevertheless, by virtue of the nature of the problem of Sudoku, this algorithm isn't applicable in this context. A* is designed for problems with a clear start and end state with paths, whereas Sudoku is about combinations of cell-values rather than pathfinding.

2. 2. 2 Adversarial Search

Adversarial search is a specialized search strategy for decision-making in competitive scenarios with multiple agents or players. It utilizes minimax and optimization algorithms to compute the best possible move to make against an unpredictable opponent. In chess, for instance, you play against an opponent, and each move has an impact on the probability of you winning. Although an opponent's moves are unpredictable, programs are able to analyze the optimal move for you to perform in favor of your chance of winning. Ultimately, there is no opposing agent making moves against you in Sudoku. You are the only one providing input to the board, and you are essentially checking your own previous moves rather than monitoring someone else's. Therefore, this type of searching strategy is an irrelevant search strategy for Sudoku.

2. 2. 3 Depth First Search and Backtracking

Depth First Search is a recursive approach to traversing graphs and tree-like data structures. It explores as far down a branch as possible before hitting the base case and backtracking. Although this approach of exploring all nodes doesn't solve the combinatorial constraint satisfaction problem, it possesses some of the previously discussed qualities needed in order to solve a Sudoku puzzle. A few similarities of depth first search and backtracking are:

- The incremental build of partial solution: *Through recursive traversal, a new node is appended to the output for each call. It's worth noting that in DFS the act of appending an element is guaranteed to remain in the final output of the algorithm. In contrast, backtracking algorithm may undo that decision and replace it with another variable later on*
- Base case: *Both algorithms have recursive base cases. However the difference is that DFS retreats at leaf nodes whereas backtracking's base case is dependent on the numerical variable constraints*

Although the algorithms overlap in certain areas, they do have many substantial differences. DFS explores all nodes, whereas backtracking focuses on the relationships or constraints to each candidate. Backtracking prioritizes variable values while DFS goes as deep as possible before retreating, and the traversal order is defined by the positions of the elements in the data structure. In conclusion, applying DFS to solve Sudoku without any structural and conditional modifications would not be feasible, as Sudoku is a CSP and is by extension aiming for a combination of values as a solution.

2. 2. 4 Local Search

Local search focuses on finding solutions by iterative improvement, and is appropriate for optimization problems and CSPs as a result of its nature of using a local subset of the search space to iteratively optimize solutions until the optimum is reached. The iterative optimization procedure is predicated on neighbors. For each solution, a neighborhood is defined, which is a collection of all possible solutions that can be reached by slightly adjusting the current state. All neighbors are then evaluated and compared to the current state as a potential for optimizing the current solution. This iteration ends when no better neighbor is found, ensuring that a local optimum has been reached. This approach makes sense in the context of Sudoku. The iterative optimization of local search algorithms is equivalent to incrementally building a partial solution and performing minor adjustments to refine the current state. With that said, turning the constraint satisfaction problem of Sudoku into an optimization problem is a possibility.

3. Approaches

This section first covers the two concepts “backtracking” and “constraint propagation” separately and then dives into the final approach where both concepts are combined into one algorithm suitable for the CSP of Sudoku. These approaches are based on two search strategies described in the latest section. Backtracking is a variant of Depth First Search due to its recursive node-expansion structure, whereas Simulated Annealing is categorized as a Local Search as it turns the CSP into an optimization problem.

3. 1. Backtracking

This is a searching technique that aims to, in Sudoku, to assign each cell with numerical values in accordance with the given constraints or rules of the board game. Of course, the algorithm cannot predict all of the different combinations for a given puzzle, and must therefore verify the validity of a combination, and if it fails, it must also identify the error. Thus, the signifying feature of backtracking is its undoing of previous decisions to return to a prior decision point. Keeping the CSP constraints in mind, as well as this error-prone combinatorial approach in mind, the following questions must be addressed:

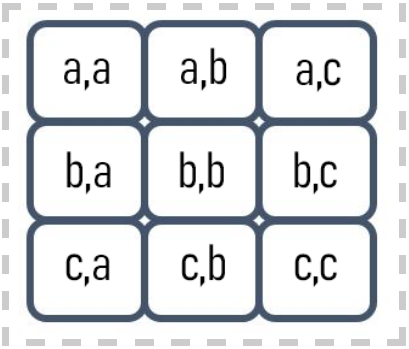
- *Is it the current candidate that causes the error? Can it be fixed by just changing the value of this candidate?*
- *Are all candidates taken? If no combinations are valid, the algorithm must backtrack and modify the partial solution to correct the erroneous candidate. If so, exactly which candidate or candidates must be changed, with respect to the direct and indirect constraints?*

It's important to have a fundamental understanding of the concept of backtracking and how it uses recursion along with an incremental addition to a partial solution to fully solve a Sudoku puzzle. In essence, for each cell-value **X**, where **0 < X < 10** that is added to the partial solution, the following three checks must always be performed:

- *Is another **X** appearing in the same row?*
- *Is another **X** appearing in the same column?*
- *Is another **X** appearing in the same 3x3 subgrid?*

3.1.1 Backtracking General Cases

Below, cases of conflicts of backtracking will be demonstrated. These conflicts cause the algorithm to retrace and revert to try new combinations to satisfy the given constraints. This demonstration is divided into sections or categories of general cases encountered in Sudoku. Cases such as *Lone Single*, *Hidden Single*, *Naked Pair* and *Hidden Pair* are involved in these categories but not directly mentioned or referred to. For reference, the subgrid in the picture below will be used for explanations.



General Conflict 1 - Trying different candidates for a partial solution

Backtracking is founded on retracing. If at least one direct constraint is invalid, it implies that adding a new candidate **X** results in a flawed Sudoku board and the algorithm must retrace old mistakes. It's important to understand that only removing or replacing the **X** in this case would not guarantee a solid partial solution. Chances are that replacing **X** with **Y** at position **[a, b]** also is invalidated because another **Y** already appears at coordinate **[c, b]**, indicating a shared column of duplicate values. In addition, if you for instance replace **X** with **Z** at an independent position **[c, c]**, the candidate may still be invalid, if another **Z** appears in any cell in the shared 3x3 subgrid.

General Conflict 2 - Simple modification

On the contrary to the aforementioned general conflict, there may be cases of invalidation that aren't deeply rooted. This conflict arises when all variable values of the domain aren't yet inserted on the Sudoku board. Thus, it usually occurs in the initial phases of the execution of the algorithm. For example, using the previous example where **X** is the invalid candidate, simply replacing it with **Y** such that no other conflicts arise would imply that **Y** is compatible with the current partial solution, and ultimately the algorithm moves to the next cell.

General Conflict 3 - Deep recursive modifications

As more and more cells are filled with numbers, more and more indirect constraints are applied. In parallel, partial solutions grow and approach the state of becoming the final and valid solution for the Sudoku board. However, as the board's cells are populated, each new candidate must align with all of the existing constraints in this CSP problem. This elevates the probability of a variable assignment being invalid and having to backtrack to fix an earlier assignment in partial solutions. It's also worth noting that, towards the end, as partial solutions nearly amount to all of the 9x9 cells on the board, chances are that the algorithm has to retrace deeper in order to find and fix the error. For example, let's assume that **X** is the last and 9th cell-value that must be placed at the exact position of **[c,c]**, such that all of the other 8 cells are already in place in the 3x3 subgrid and part of the partial solution. This scenario would entail that **X** has to be the candidate to fully solve the subgrid, such that the algorithm can't swap **X** for another value to be the candidate. If another cell appears in another subgrid but on a shared row of the 9x9 board with value **X** (position **[c, b]** for instance), it implies that we can't insert **X** to the subgrid with one remaining value. Now, the algorithm has found itself in a deep recursive problem. Given the context of the presented situation, it cannot try any other candidate on the subgrid, and can therefore conclude that the error is rooted in one or multiple other previously assigned cells. At this point, the backtracking algorithm will undo a multitude of decisions to identify the root of the problem. However, this is not as simple as the two general conflicts or cases covered earlier. In this scenario, it must systematically revert previous variable assignments to try new ones.

3. 2 Constraint Propagation

If Sudoku is a combinatorial problem, the second most important issue, besides actually retrieving the combinations, is to minimize the number of combinations to consider, since when there are too many combinations or branches, the time and space complexity gets worse. At its core, this type of propagation is about ignoring redundant branches or certain combinations to improve the algorithmic efficiency. To do this, we need to separate between the branches that have a chance of contributing to the final output, and the ones that impossibly can yield a valid combination.

3.2.1 Pairwise Constraints

In the broad context of CSPs, pairwise constraints are used to improve constraint propagation by highlighting relationships between pairs of variables. It's a technique for reducing the domain of variables, and in this way achieve a higher efficiency. In cases where pairwise constraints between two variables indicate restrictions of valid assignments, those values are then removed from the domain. For instance, in Sudoku, if "8" and "6" appears in the first row, we discard these values from the domain of values for empty cells in that row.

3.2.2 Minimum Remaining Values Heuristic

As addressed in the section above, the objective of pairwise constraints is to reduce the domain of possible values for cells. The Minimum Remaining Values heuristic is about selecting the cell with the least remaining values in its domain. Intiutively, this approach of prioritizing the constrained cells makes sense, because these are the ones that are most likely to cause violations as the Sudoku game progresses. Conversely, the cells with a broader domain are more flexible as they aren't yet bounded by the indirect constraints. With that said, this heuristic is responsible for the initial variable selection, but it is dependent upon constraint propagation and pairwise constraints to reduce the cell-domains.

3.3 Combined: Backtracking & Constraint Propagation

Ultimately, the implementation approach to Sudoku that was used in this assignment was a combination of backtracking and forward checking. In this section, as both concepts already have been covered separately, the relationship between the resulting efficiency and the variable assignments that constraint propagation brings will be examined. Keeping the recursive nature of backtracking in Sudoku and the continuous application of constraint propagation in mind, expressing the resulting efficiency of the propagation as a relationship is possible:

- ***N** is the n:th filled in cell in the Sudoku board, representing the X-value mathematically*
- ***Y** is a measure of the number of operations required to solve a Sudoku board. The higher the value, the more inefficient the algorithm is*

3.3.1 Analysis of Y as N grows

This section addresses the impact that constraint propagation has on backtracking, and uses mathematical functions to highlight the relationships and their implications. These functions are abstractly defined and only include the most important factors. First, the original backtracking algorithm's performance without any application of constraint propagation can be abstractly defined as:

$$b(n) = n * Nb * Nr$$

Where..

- ***n*** *is the current number of assigned cells (1=< n <= 81)*
- ***Nb*** *is the number of backtracks performed up until the n:th filled in cell*
- ***Nr*** *is the number of recursive iterations performed up until the n:th filled in cell*
- ***b(n)*** *is the backtracking algorithm without constraint propagation, and its value represents the operations required*

As already discussed at length, the purpose of constraint propagation, in this assignment, is to enhance the backtracking algorithm's efficiency. Breaking this down on a mathematical level, this implies that the new function that represents the combination of backtracking and propagation must require less operations to solve the Sudoku board. The new function **c(n)** must therefore equate to **b(n) - A**, where **A** is the number of reduced operations:

$$c(n) = b(n) - Nc$$

Where..

- ***b(n)*** *is the backtracking algorithm's performance without constraint propagation*
- ***Nc*** *is the number of operations eliminated or propagated, as a result of combining constraint propagation with* ***b(n)***
- ***c(n)*** *is the performance of backtracking combined constraint propagation algorithm*

In order to fully understand this relationship, we need to bring to attention the improvements and benefits that come with combining both concepts. Below, the new function is summarizing the yielded efficiency of the combined approach in contrast to the backtracking approach:

$$i(n) = b(n) - c(n)$$

Where..

- ***i(n)*** *is the improvement achieved by applying constraint propagation to the backtracking algorithm*

Conditions that are always true..

- $b(n) \geq c(n)$
- $b(n) - c(n) \geq 0$
- $i(n) \geq 0$
- $i(n) = Nc$

3.3.2 Identified trend of Y as N grows

As mentioned in the “Theoretical Background” section about CSPs, indirect constraints incrementally build in parallel with the number of currently assigned variables in the partial solution. Thus, for each cell filled, or for each increment of the **n:th** filled in cell, the number of backtrackings increases disproportionately:

- The **n:th** cell must perform backtracking with respect to all previous cells inserted as **{ n∈Z ∣ 1 ≤ x ≤ (n-1) } th** and their respective indirect CSP constraints

- The **(n + 1)th** cell has **{ n∈Z ∣ 1 ≤ x ≤ n }** constraints currently active in the Sudoku board

Conclusively, as each new cell adds indirect constraint to all already-existing cells, a disproportionate increase tied to the size of **n** is present. This disproportionate increase entails that the approach of combining backtracking and forward checking doesn’t have a linear impact, but an exponential one. In the “Results” section, this trend will be observed further.

3. 2 Simulated Annealing

As discussed in the theoretical background, two types of search strategies that are applicable to Sudoku considering its combinatorial nature, are depth-first search with a few backtracking tweaks and local optimization search. Simulated Annealing is a type of local search that is based on physical annealing, where the temperature is elevated at first and then gradually decreases. As the temperature is cooling down, there is a possibility to explore energies to find the global optimum.

3.2.1 Temperature

Whether it be in physical metallurgy or in an algorithmic context, the temperature component of annealing is imperative. For algorithms, it governs the probability of accepting worse solutions. The higher the temperature, the more the solution space will be explored. As the temperature gradually is reduced over the lifetime of the algorithm, the chances of accepting worse solutions decreases. Moreover, the temperature serves as a boundary or standard for the quality of the solution. Hence, if the temperature always is high, indicating an exploration period, then the standard for accepting solutions that brings the current one closer to the final CSP solution won’t be performed, and the algorithm will as a result continuously accept worse and worse solutions. Conversely, if the temperature is always low, the exploration phase will be non-existent. However, this contradicts the very nature of CSPs. Recall the combinatorial necessity of a CSP and how it enforces an initial phase in algorithms to explore potential variable assignments before refining it into the final solution. Therefore, the conclusion is that the temperature must start at a high value and then gradually decrease, and

that it wouldn't work if it were the other way around, such that it starts low and goes high. Because if the algorithm starts with setting high standards and rejects most solutions, it will then keep rejecting them as the search space or temperature increases without actually having a look at their potential.

3.2.2 Neighboring Solutions

As explained in the background section about local search strategies, neighboring solutions is a common concept. In Simulated Annealing, they are incorporated together with the temperature element described in the section above, such that they are minor modifications of the current solution and serve as alternatives. These alternatives are evaluated in terms of conflicts and the tolerance of the current temperature, using the energy function. If a neighboring solution is evaluated as high energy, search exploration takes priority over accepting a better solution, and the probability for this is:

$$P = e^{-\Delta E/T}$$

- ***ΔE:*** *The change in energy*
- ***T:*** *Current temperature*

3.2.3 Behavior similar to backtracking

In Sudoku, or in CSPs in general, as the solution is a combination of variable assignments that are all aligned with the given constraints, the main challenge that algorithms are tasked with is the unpredictability that comes with these types of problems. This unpredictability forces the algorithms to try partial solutions against the problem-constraints, and then using the feedback of the conflicts to construct a more accurate partial solution. By extension, these algorithms resemble refining processes where the end-goal is to refine the partial solutions so much that eventually all conflicts are eliminated, leaving us with the final and complete solution to the CSP. As covered earlier in this report, the "Backtracking and Constraint Propagation" approach used backtracking to revert the conflicts of partial solutions. The question that now naturally emerges is:

- *How does the algorithm of Simulated Annealing incorporate the concept of refining or reverting solutions to solve a CSP problem?*

Taking into consideration what was addressed in the previous section about the temperature cooling down effect coupled with neighboring solutions, we can see that this resembles a refining process. Opting for one solution over another based on the very metric of constraint conflicts is essentially what backtracking is about. The difference between the two approaches is:

- ***Backtracking:*** *Reverts to other cell-values via recursion*

- ***Simulated Annealing:*** *Evaluates entire neighboring solutions and compare their conflicts to the current one*

More specifically, this refinement process serves the purpose of escaping the local optimum, the best solution in a subregion of the CSP. This is done in the initial phases of the algorithm when the temperature is high and worse neighboring solutions are accepted such that it "moves backwards" or "backtracks" to try other options that seem less promising but have potential to be a better overall solution.

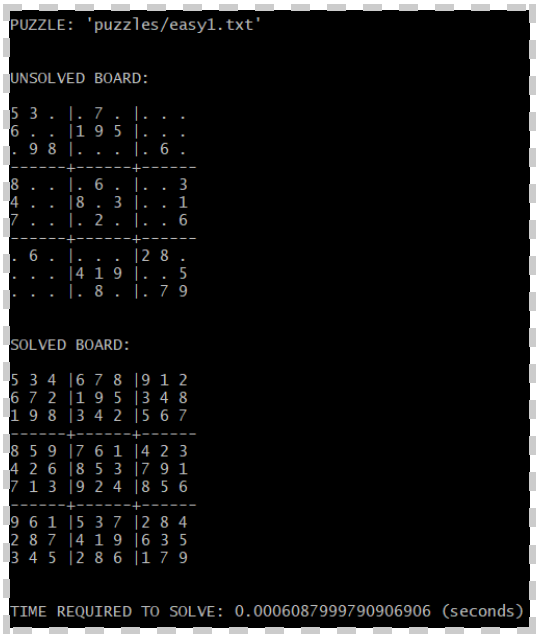
4. Results

This section is solely intended for presenting the results briefly. The rationale and conclusions derived from these insights coupled with the approaches and theoretical background will be presented in the next section "Conclusions".

4. 1 Solution Input/Output

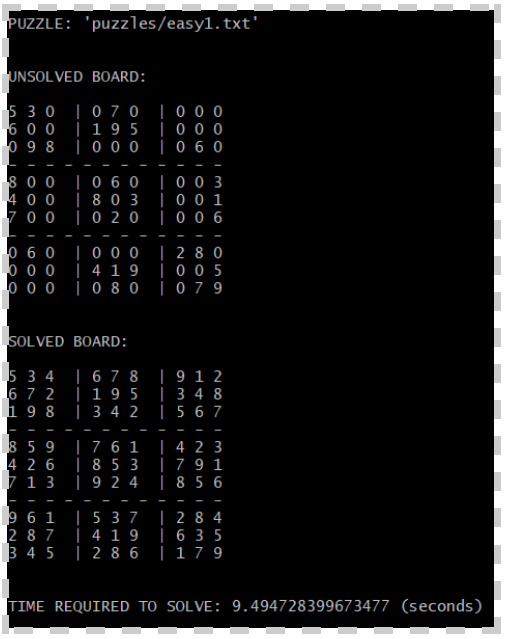
Approach 1:

Backtracking & Constraint Propagation:



Approach 2:

Simulated Annealing



4. 2 Performance

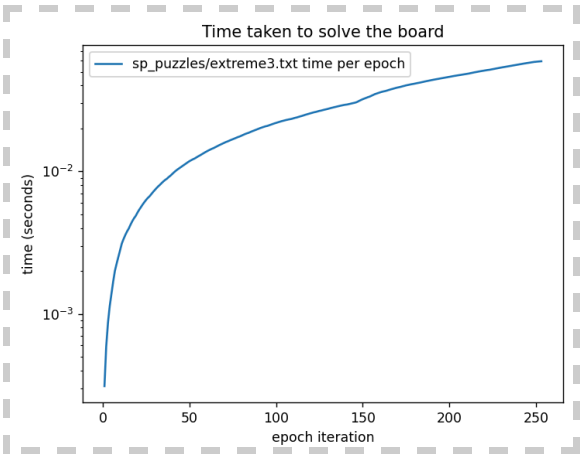
One general pattern that is clearly visible across all graphical pictures displayed below is the exponential function that was presented in the "Approach" section. For each epoch iteration

(X-value), the required time (Y-value) increases, meaning that the number of operations (backtracking or simulated annealing escaping local optimum depending on the algorithm) increases. Keep in mind that for each epoch iteration, the algorithm gets one step closer to solving the board. This ties back to the concept explained at length in the previous sections, in regards to the increased number of applied indirect constraints as the partial solution grows. Ultimately, this results in a decreased probability of a new arbitrarily picked candidate being valid.

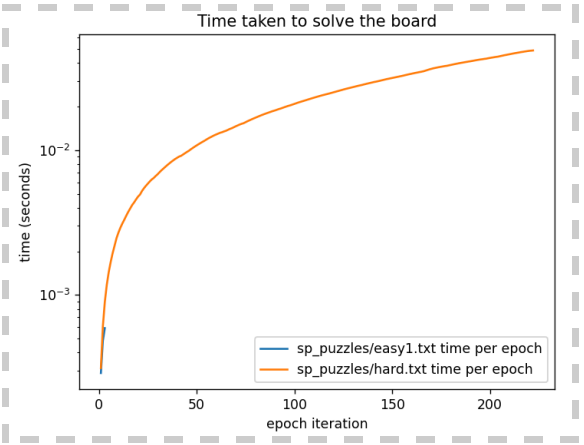
4. 2. 1 Backtracking & Constraint Propagation

In the three visualizations below, the epoch iteration is equivalent to the recursive calls, since it's invoked as a propagation function at the start of each call in the code implementation:

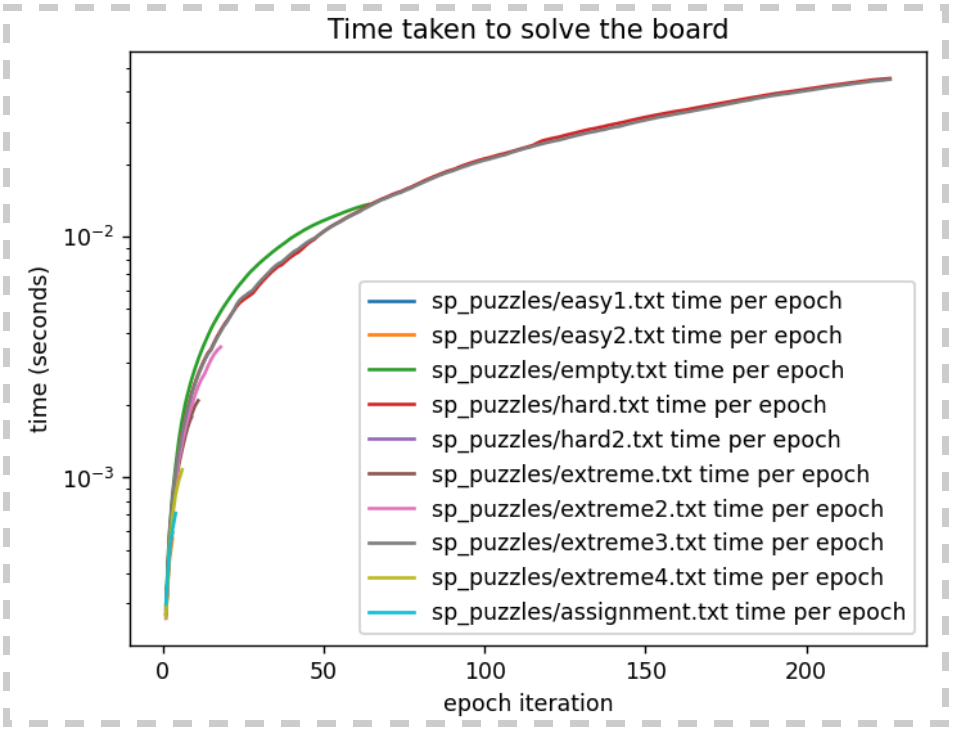
Solving 'extreme3.txt' puzzle



Solving 'easy1.txt' and 'hard.txt' puzzles



Solving multiple puzzles



In summary, the three pictures above look almost identical despite running different puzzles. This is due to the exponential pattern of increased operations and resources for each epoch iteration, and as explained in the "Approach" section, the cause for this is that the indirect constraints force more recursive calls to correct the mistakes of cell assignment. To further confirm this, I implemented functionality for writing values obtained in the backtracking algorithm during runtime to 2 JSON files:

- ***recursive-data.json:*** *Tracks the recursive iterations and backtracks performed when solving each puzzle*
- ***backtrack-timeline.json:*** *Maps the i:th recursive iteration to the number of backtrackings for a selected puzzle*

recursive-data.json

performances > recursive-data.json > ...

```
{
    "recursiveIterations": {
        "puzzles/easy1.txt": 0,
        "puzzles/easy2.txt": 0,
        "puzzles/empty.txt": 47,
        "puzzles/hard.txt": 113,
        "puzzles/hard2.txt": 0,
        "puzzles/extreme.txt": 2,
        "puzzles/extreme2.txt": 5,
        "puzzles/extreme3.txt": 113,
        "puzzles/extreme4.txt": 0,
        "puzzles/assignment.txt": 0
    },
    "backtrackings": {
        "puzzles/easy1.txt": 0,
        "puzzles/easy2.txt": 0,
        "puzzles/empty.txt": 0,
        "puzzles/hard.txt": 54,
        "puzzles/hard2.txt": 0,
        "puzzles/extreme.txt": 0,
        "puzzles/extreme2.txt": 2,
        "puzzles/extreme3.txt": 54,
        "puzzles/extreme4.txt": 0,
        "puzzles/assignment.txt": 0
    }
}
```

backtrack-timeline.json

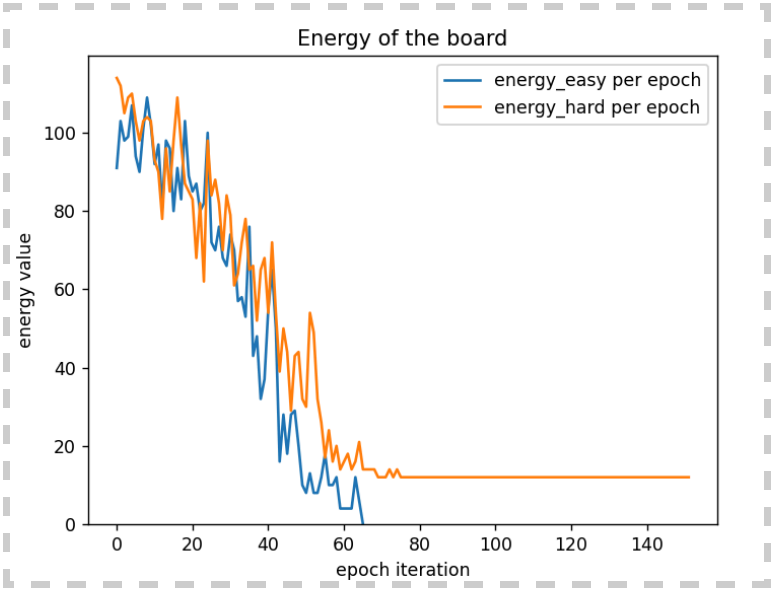
performances > backtrack-timeline.json > ...

```
{
    "9": 1,
    "10": 2,
    "11": 3,
    "12": 4,
    "13": 5,
    "15": 6,
    "17": 7,
    "19": 8,
    "20": 9,
    "24": 10,
    "25": 11,
    "26": 12,
    "30": 13,
    "31": 14,
    "32": 15,
    "33": 16,
    "37": 17,
    "39": 18,
    "40": 19,
    "43": 20,
    "45": 21,
    "46": 22,
    "49": 23,
    "50": 24,
    "51": 25,
    "54": 26,
    "56": 27,
    "58": 28,
    "59": 29,
    "60": 30,
    "64": 31,
    "68": 32,
    "69": 33,
    "72": 34,
    "73": 35,
    "75": 36,
    "76": 37,
    "77": 38,
    "puzzle": "puzzles/extreme3.txt"
}
```


4. 2. 2 Simulated Annealing

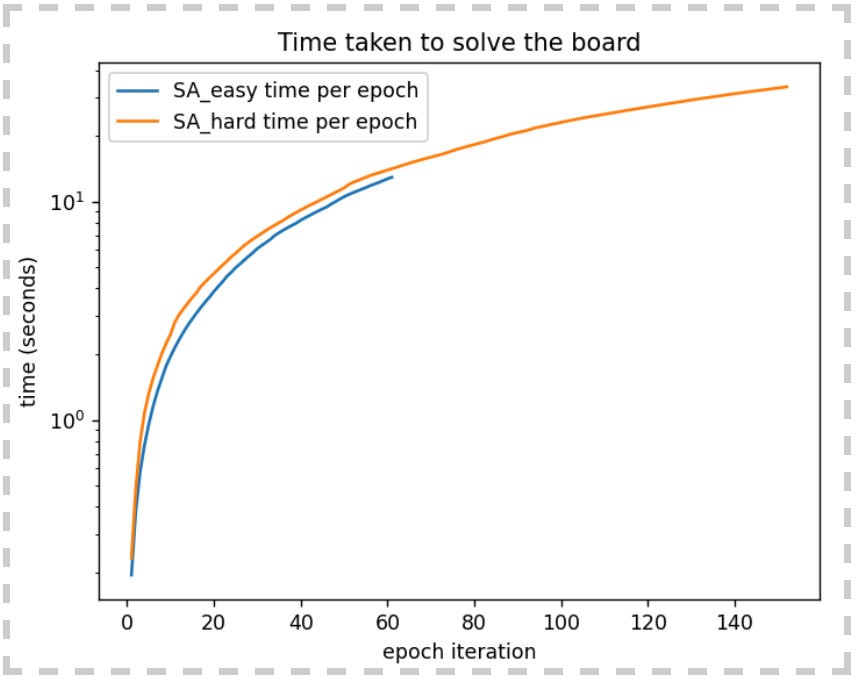
Two puzzles, one easier (easy1.txt) and one harder (hard1.txt), are plotted together in the visualizations below.

Display the energies during the algorithm's lifetime:



Recall from what was mentioned in the “Approach” section about energies and temperatures, and note that the cooling process is manifested. The annealing starts at a high temperature to explore solutions and gradually decreases for optimization.

In the same 2 puzzles, the following time analysis result was obtained:



Notice that the exponential slopes of the picture above are identical to the ones illustrated for the backtracking algorithm.

4. 3 Table Average Approximations

Below, two tables are illustrating the respective approaches. The displayed data is based on the average obtained from 5 sample runs, and the execution time is measured in seconds.

Backtracking & Constraint Propagation

Puzzle	**Sample Runs**	**Execution Time** *(min, max, avg)*	**Success Rate**
easy1.txt	5	0.00059, 0.00065,0.0062	100%
hard1.txt	5	0.008, 0.055, 0.027	100%

Notice that the minimum and maximum times are close to each other, implying that the number of operations using this approach is consistent. Also, pay attention to the fact that the algorithm solves the puzzles in only a matter of split seconds.

Simulated Annealing

Puzzle	Sample Runs	Execution Time *(min, max, avg)*	Success Rate
easy1.txt	5	8.94, 30.37, 17.82	100%
hard1.txt	5	27.26, 30.1, 28.88	100%

In contrast to what was shown in the previous table, the difference between *min* and *max* time is much more significant for Simulated Annealing. In addition, much more is time required to solve the puzzles when applying this algorithm.

5. Conclusions

After exploring both approaches and examining their results, it's now time to reflect about what was discovered. Each major finding is categorized as its own subsection below:

5. 1 All CSPs require an initial exploration phase

As it clearly appears in Simulated Annealing, the temperature must be high in the beginning to explore neighboring solutions with less potential, as they still have the chance of satisfying the combinatorial solution completely. Turning this process around such that the temperature is initially low would only reject all solutions, resulting in not finding the best possible CSP solution. Similarly, the backtracking approach adapts to an initial exploration phase. To "Move backwards" or reverting is what resembles this phase, but rather than gradually decreasing the probability like Simulated Annealing, the probability is instead depending on the constraint propagation. Ultimately, the only way for the algorithm to progress towards the optimized goal is to make mistakes in the exploratory period and correct them accordingly. Rationalizing the natural structure of CSPs, I'd argue that this initial searching phase is essential for solving any kind of CSP problem. Since these types of problems offer a degree of unpredictability that requires computational trial and error of variable assignments, I'm concluding that this period of trial and error is the exploration phase. Without first exploring variable assignments and testing the validity of those with respect to the given constraints, I think it's impossible for an algorithm to predict the assignments on the first few attempts.

5. 2 Exponential relationship between epoch and time required

As established in the "Approach" section, when applying constraint propagation to the backtracking algorithm, the impact on efficiency **(Y)** that it yields is exponential to the number assignment variables in the partial solution **(N)**. This relationship was also clearly

visible in the charts in the “Results” section. Now, this exponential relationship appears in the recursive nature of backtracking, but what about Simulated Annealing?

Well, to begin with, we can observe in the last picture in the “Results” section that the graph is of the exact same exponential shape as the ones presented for the backtracking approach. Thus, we can conclude that the exponential relationship is in fact present in Simulated Annealing as well. Let’s now dig deeper into this and understand why and how this exponential trend appears in both CSP problems. Perhaps this trend of algorithmic progress applies to all CSPs, given their unique combinatorial structure?

Since Simulated Annealing is about optimization as opposed to recursive reverting on-the-go, there are some differences in comparison to the backtracking approach:

- *<u>High temperature:</u> Indicates many accepted neighboring solutions and less emphasis on constraints and conflicts. In the initial phases of the backtracking approach, a similar concept is executed, where all first-attempt candidates are accepted because the board has yet to apply the constraints that deny partial solutions yet*

- *<u>Low Temperature:</u> Indicates many rejected neighboring solutions and more emphasis on constraints and conflicts. In the later phases of the backtracking approach, it “optimizes” its proposed candidates (or neighboring solutions in SA approach) by checking and reverting conflicts*

Conclusively, acknowledging the fact that the temperature is always initially high and then gradually decreases, and the general similarities in the backtracking approach listed above, it appears that both approaches have an exponential relationship between **N** and **Y**. Looking at this from a broader perspective, and the unpredictable combinatorial aspects of CSPs, a few general conclusions can be drawn:

- *The only logical way to solve CSPs is to establishing two phases: an initial <u>exploratory phase</u> and a terminating <u>optimization phase</u>*

- *The exploratory phase puts less emphasis on conflicting constraints and assigns variable values to explore the space, and therefore require less epochs or operations*

- *The optimization phase puts more emphasis on finding the best final solution, and is therefore willing to revert previous assignments and accepts only very few options*

- *During the execution of the algorithm or approach, as the exploratory phase gradually transitions into the optimization phase, an exponential relationship between **N** and **Y** is established*

5.3 Simulated Annealing vs Backtracking & Constraint Propagation

In this section, the two approaches will be evaluated and compared in accordance to the 4 criteria *completeness*, *optimality*, *temporal complexity* and *spatial complexity*, based on their performance observed in the section "Results".

- **Completeness:** As the table in "Results" indicates, both algorithms consistently find solutions if they exist for all of the sample runs with a ratio of 100%. Hence, we can infer that they are complete.

- **Optimality:** To be clear, the optimal solution discussed in this section refers to the one and only complete solution out of a set of all possible complete solutions that can be reached the fastest, obtaining the best temporal and spatial complexity.

    - Simulated Annealing: Even though it finds complete solutions to a problem, it does not always find the optimal one, in the sense that the exploration phase is to an extent governed by a randomized probability. This is supported by the table in the "Results" section, where an enormous difference between the min and max execution time is shown. For the *easy1.txt* puzzle, the min time was 8 seconds, whereas the max took 30 seconds.

    - Backtracking & Constraint Propagation: Due to the deterministic and structural approach of this algorithm, unlike Simulated Annealing, it is very consistent. Looking at the table in the previous section shows that both puzzles were solved within just a matter of split seconds, and that the min and max times also were differentiated by a fraction of seconds.

- **Temporal Complexity:**
    - Simulated Annealing: This approach relies on random sampling of neighboring solutions coupled with a varying temperature-probability for solution exploration. In turn, this causes varying runtime durations, since the proportion of high and low temperatures is not optimally balanced to the conditions of the given problem, leaving the algorithm stuck in local optimums for extended periods of time. It isn't until the temperature is adjusted that the local optimum is escaped such that the algorithm can make progress towards the global optimum, or the complete solution. These highly varying runtime durations can be observed in the table in the "Results" section. For the *easy1.txt* puzzle, the min time was 8 seconds, whereas the max took 30 seconds. In addition, the average time for solving *hard1.txt* is as much as 28.88 seconds. We can thus conclude that, considering a problem bounded by a fixed size of 9x9 board, the temporal complexity is bad. Imagining this algorithm applied to other CSPs that involve more combinations makes you realize how time-consuming this approach can be when used at a larger scale.

    - Backtracking & Constraint Propagation: Due to its deterministic and structural qualities, this approach enables immediate feedback to propagate branches early. By extension, this counteracts the case of Simulated Annealing where the algorithm gets stuck in a local optimum. Thus, the backtracking approach has a great temporal complexity in comparison to Simulated Annealing. In the "Results" section, the puzzle *hard1.txt* was fully solved within 0.027 seconds on average, making it the most efficient approach by far.

- **Spatial Complexity:** Similar for both approaches is that they must allocate memory to maintain the state of the 9x9 board. On the other hand, the backtracking approach has to keep track of the recursive call stack as well. The depth of this stack is what determines the memory needed, and it varies during the runtime of the algorithm, but theoretically speaking, the maximal depth, or recursive calls that can be stored in Sudoku is 9x9. As for Simulated Annealing, it must allocate additional memory for the current best solution and maintain neighboring solutions with their associated energy-evaluation. Moreover, the neighbors are dropped from memory after each temperature shift, just like recursive calls in the backtracking approach are dropped as the retraced mistakes are resolved.